

UBRZANJE ALGORITMA ZA ODREĐIVANJE GRADIJENTA I DETEKCIJU IVICA NA SLIKAMA

1. Opis sistema

Potrebno je realizovati sistem kojim se omogućava određivanje gradijenta i detekcija ivica sa ulaznih slika. Sistem se sastoji iz procesorskog dela (Zynq7), DMA kontrolera, AXI Timer komponente i komponente akceleratora detekcije ivica (**acc_edge_detection**).

Ulazna slika (**img**) je zadata u **uint8** formatu i nalazi se u binarnom fajlu na **host** računaru. Slika se može preneti u DDR memoriju na razvojnom sistemu korišćenjem **xdb** komande **mwr**. Nakon toga potrebno je odrediti vrednosti horizontalnog i vertikalnog gradijenta, magnitude gradijenta kao i pozicije ivica na osnovu zadate vrednosti praga. Detalji algoritma su navedeni u odeljku 2.

Računanje se najpre obavlja softverski korišćenjem funkcije **EdgeDetectionSW** implementirane na procesorskom delu sistema. Ulazni argumenti funkcije su pokazivač na ulaznu sliku **DataIn**, pokazivač na izlazni bafer **DataOut** kao i parametri procesiranja koji se sadrže u strukturi **EdgeDetectionParams**. Ova struktura ima polja koja definišu širinu i visinu ulazne slike (**ImgW**, **ImgH**), vrednost praga (**EdgeThr**), mod rada (**Mode** – definiše koji rezultat se čuva u izlaznom baferu), način tretiranja ivičnih elemenata (**Border**) kao i parametar zaobilaženja celokupnog procesiranja modula (**Bypass**) pri čemu se u tom slučaju na izlaz prosleđuju neizmenjeni pikseli ulazne slike.

Procesiranje se može ubrzati korišćenjem komponente hardverskog akceleratora. Potrebno je realizovati funkciju **EdgeDetectionHW** kojom se omogućava korišćenje hardverskog akceleratora **acc_edge_detection**. Komponentu akceleratora je potrebno realizovati u okviru projekta prema specifikacijama koje su date u narednim odeljcima. Argumenti funkcije su identični kao i argumenti funkcije kojom se realizuje softversko procesiranje. U okviru funkcije je potrebno konfigurisati komponentu akceleratora kao i komponentu DMA kontrolera. Komponentu DMA kontrolera konfigurisati tako da se elementi ulazne slike šalju preko AXI Stream interfejsa u modul akceleratora i da se izlazni podaci iz akceleratora prihvataju preko AXI Stream interfejsa i upisuju u memoriju izlaznog bafera.

Korišćenjem komponente AXI Timer-a odrediti vreme izvršavanja funkcija **EdgeDetectionSW** i **EdgeDetectionHW** u mikrosekundama. Uporediti brzine procesiranja i odrediti faktor ubrzanja koje se postiže korišćenjem modula hardverskog akceleratora.

Implementirati funkciju **CheckData** kojom se verifikuje poklapanje rezultata softverskog i hardverskog procesiranja.

Rezultat procesiranja snimiti u odgovarajući fajl na host računaru, korišćenjem **mrd** komande, i prikazati ga korišćenjem datih **Python** skripti. Uporediti rezultate softverskog i hardverskog procesiranja sa rezultatim procesiranja u Python-u i verifikovati ispravnost primenjenog pristupa.

Omogućiti da se, bez ponovnog pokretanja programa, nakon završetka procesiranja trenutne slike pređe na procesiranje naredne slike sa novim parametrima procesiranja. Demonstrirati funkcionalnost sistema testiranjem različitih parametara procesiranja kao i različitih ulaznih slika.

2. Opis algoritma

Kako bi se detektovale ivice potrebno je najpre odrediti vrednosti horizontalnog (**gradH**) i vertikalnog (**gradV**) gradijenta. Vrednosti gradijenta se određuju primenom standardnog gradijentnog operatora i predstavljaju celobrojne označene 8 bitne brojeve. Magnituda gradijenta (**gradM**) se određuje aproksimativno kao zbir apsolutnih vrednosti horizontalnog i vertikalnog gradijenta i predstavlja celobrojne neoznačene 8 bitne brojeve. Vrednosti ivica se dobijaju poređenjem magnitude gradijenta sa zadatom vrednošću praga (**edgeThr**) i mogu imati jednu od dve vrednosti: 0 (nije detektovana ivica) i 255 (detektovana je ivica). U nastavku su prikazani izrazi za određivanje prethodno opisanih elemenata pri čemu su redovi i kolone slike označeni sa **r** i **c** respektivno:

- 1) Horizontalni gradijent:

$$\text{gradH}[r, c] = \frac{\text{img}[r, c + 1] - \text{img}[r, c - 1]}{2}$$

- 2) Vertikalni gradijent:

$$\text{gradV}[r, c] = \frac{\text{img}[r + 1, c] - \text{img}[r - 1, c]}{2}$$

- 3) Magnituda gradijenta:

$$\text{gradM}[r, c] = \text{abs}(\text{gradH}[r, c]) + \text{abs}(\text{gradV}[r, c])$$

- 4) Detektovane ivice:

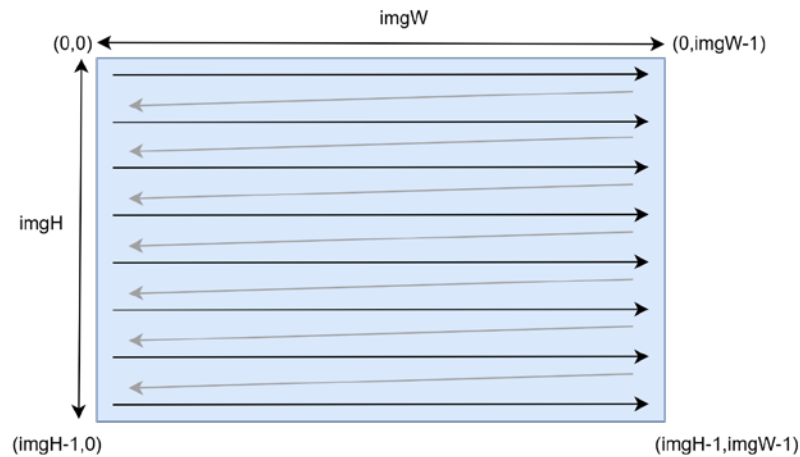
$$\text{edge}[r, c] = \begin{cases} 0, & \text{gradM}[r, c] < \text{edgeThr} \\ 255, & \text{gradM}[r, c] \geq \text{edgeThr} \end{cases}$$

3. Ivični uslovi

Izlazne matrice imaju iste dimenzije kao i ulazna slika. Obratiti pažnju da se za granične vrednosti redova i kolona $r = 0, r = \text{imgH} - 1, c = 0$ ili $c = \text{imgW} - 1$ može desiti da neki od članova koji su potrebni za izračunavanje **gradH** i **gradV** imaju indekse koji izlaze van okvira slike (npr. pri računanju **gradH** za $c = 0$ drugi član ima vrednost $\text{img}[r, -1]$). Jedno rešenje je da se za elemente koji ispadaju van okvira slike usvoji da je njihova vrednost jednaka 0. Ovaj pristup se koristi ukoliko je parametar **Border** postavljen na 0. Drugo rešenje je da se za ove elemente usvoji da je njihova vrednost jednaka najbližem elementu koji se nalazi u slici (npr za član $\text{img}[r, -1]$ se uzima vrednost elementa $\text{img}[r, 0]$, za član $\text{img}[r, \text{imgW}]$ se uzima vrednost člana $\text{img}[r, \text{imgW} - 1]$, itd...). Drugi pristup se koristi ukoliko je parametar **Border** postavljen na 1.

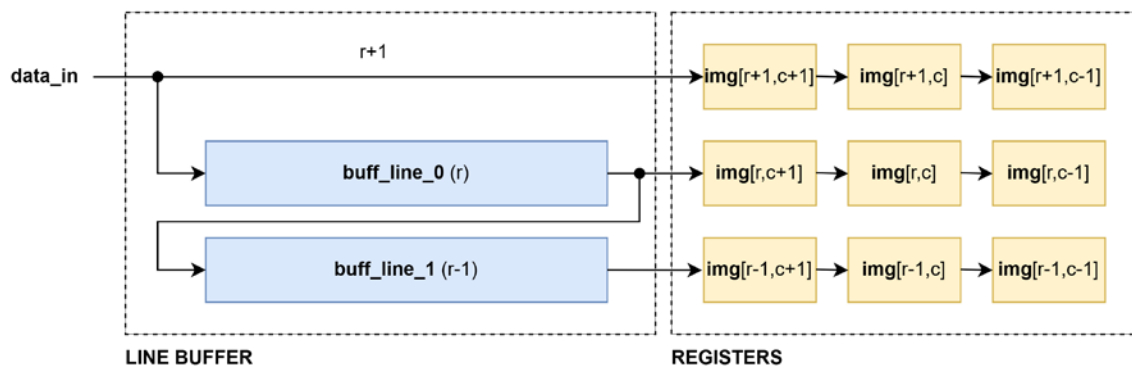
4. Arhitektura modula akceleratora

Pikseli ulazne slike se u akcelerator šalju preko AXI Stream interfejsa. Redosled slanja piksela ulazne slike je s leva na desno i prikazan je na slici 1.



Slika 1. Redosled procesiranja

Kako su za određivanje vertikalnog gradijenta potrebne vrednosti piksela iz susednih redova slike to je ove redove potrebno sačuvati u odgovarajućim baferima. Na slici 2 je prikazana struktura linijskog bafera i izlaznih registara.



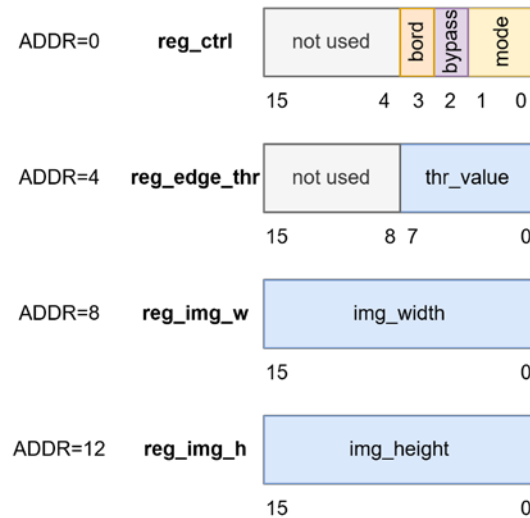
Slika 2. Realizacija linijskog bafera

Komponente linijskih bafera (**buff_line_0** i **buff_line_1**) se mogu realizovati korišćenjem IP komponente **RAM-based Shift Register** koja je dostupna u okviru Xilinx IP kataloga, što je detaljnije opisano u dodatku D1. Širina jednog reda linijskog bafera odgovara maksimalno podržanoj širini slike i u slučaju korišćenja gotovog IP modula se postavlja na 512. Ukoliko se projektuje novi blok linijskog bafera bez IP modula, maksimalna podržana širina slike se definiše *generic* parametrom **C_MAX_IMG_WIDTH** koji se može zadati prilikom instanciranja komponente. Stvarna širina slike koja se obrađuje se definiše sadržajem registra **reg_img_w** i može se menjati i nakon instanciranja komponente (u *runtime*-u). Voditi računa da se u linijski bafer upisuju isključivo validni ulazni pikseli. Zbog toga je potrebno obezbediti mogućnost da se linijski bafer zakoči ukoliko na ulazu nema validnih podataka ili ukoliko izlazni interfejs nije spreman da prihvati validne izlazne podatke.

Kako se iz svakog reda linijskog bafera može pročitati najviše 1 piksel po taktu, a u izrazima za određivanje vrednosti gradijenata su potrebni elementi iz susednih kolona, potrebno je obezbediti odgovarajuće registre u kojima će se čuvati elementi iz susednih kolona kojima je moguće pristupiti u paraleli (označeni žutom bojom na slici 2). U okviru ovih registara se nalaze svi elementi neophodni za izračunavanje rezultata gradijenata. Ukoliko se element nalazi na ivicama slike potrebno je postupiti na jedan od dva načina opisanih u odeljku 2.

5. Registri modula akceleratora

Potrebno je obezbediti da projektovani modul akceleratora ima 4 registra širine 16 bita. Adresna mapa i opis registara su prikazani na slici 3 i u Tabeli 1.



Slika 3. Prikaz registara modula akceleratora

Tabela 1. Adresna mapa i opis registara akceleratora

Adresa	Naziv registra	Opis
0x0	reg_ctrl	Kontrolni registar akceleratora. Sadržaj definisan u Tabeli 2.
0x4	reg_edge_thr	Vrednost praga za detekciju ivica. Najnižih 8-bitova definišu vrednost praga iz opsega [0,255] ostali biti registra se ne koriste.
0x8	reg_img_w	Širina ulazne slike. Zadaje se kao 16 bitni neoznačeni ceo broj.
0xC	reg_img_h	Visina ulazne slike. Zadaje se kao 16 bitni neoznačeni ceo broj.

Tabela 2. Sadržaj bita kontrolnog registra (**reg_ctrl**)

Bit	Naziv	Opis
15..4	N/A	Ne koriste se.
3	BORD	Definiše način na koji se tretiraju elementi koji figurišu prilikom izračunavanja gradijenta ali izlaze van okvira slike. 0 – Elementi koji izlaze van okvira slike imaju vrednost 0 1 – Elementi koji izlaze van okvira slike imaju vrednost najbližeg elementa koji se nalazi u slici.
2	BYPASS	Definiše da li se na izlaz samo prosleđuju elementi ulazne slike. Na taj način se preskače celokupno procesiranje akceleratora. 0 – Na izlaz se prosleđuje vrednost definisana MODE bitima 1 – Na izlaz se prosleđuju vrednosti ulaznih piksela
1..0	MODE	Definiše koja vrednost se šalje na izlaz modula: 00 – Mapa ivica (edge) 01 – Horizontalni gradijent (gradH) 10 – Vertikalni gradijent (gradV) 11 – Magnituda gradijenta (gradM))

Obezbediti da se svakom od registara može pristupiti sa procesorskog dela sistema korišćenjem AXI Lite memorijski mapirane magistrale. **Obezbediti da se vrednosti parametara**

procesiranja mogu promeniti jedino u slučaju da trenutno nije u toku proces obrade, pri čemu se u odgovarajuće registre može upisati u bilo kom trenutku.

6. Interfejsi modula akceleratora

Potrebno je obezbediti da modul akceleratora prihvata ulazne piksele i šalje izlazne rezultate preko AXI Stream interfejsa **S_AXIS_IN** i **M_AXIS_OUT** respektivno. Širina podataka (**tdata**) koji se prenose po svakom od ovih interfejsa je 8 bita. Pored interfejsa za podatke modul akceleratora implementira i AXI Lite slave interfejs **S_AXI_LITE_CFG** preko kojeg se omogućava procesorskom delu sistema da pristupi registrima akceleratorskog modula.

7. Izveštaj

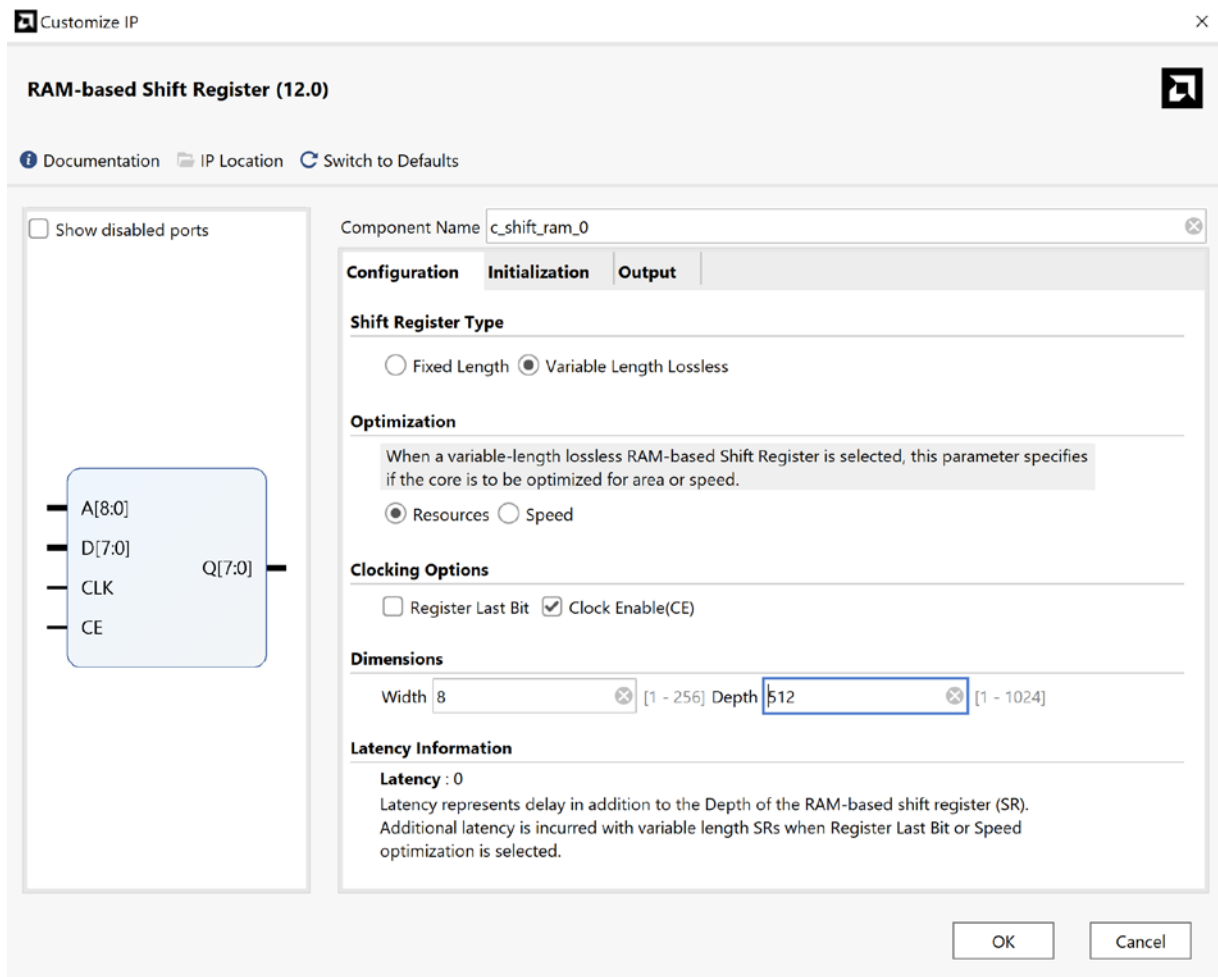
Na kraju projekta je potrebno da kreirati izveštaj koji predstavlja dokumentaciju projekta i opisuje arhitekturu sistema, realizaciju hardvera i softvera za svaki od delova projekta. Izveštaj predstavlja dokumentaciju projektovanog sistema i potrebno je da bude dovoljno detaljan kako bi neko uz izveštaj i kod mogao potpuno da reprodukuje rezultate i iskoristi delove projekta u većem sistemu. U izveštaju je potrebno opisati testiranje sistema, prikazati dobijene rezultate kao i izmerena vremena izvršavanja i postignuta ubrzanja.

Za pun broj poena potrebno je da pored potpune funkcionalnosti sistem bude smisleno particionisan i kod uredno napisan sa dovoljnim brojem komentara.

Efikasna realizacija linijskog bafera koji se generiše na osnovu parametra C_MAX_IMG_WIDTH bez korišćenja gotovih IP blokova može doneti 5 bonus poena.

D1. Korišćenje Xilinx IP komponente u VHDL-u

Linijski baferi se mogu kreirati direktno korišćenjem nizova u VHDL-u i implementacijom odgovarajuće logike pristupa, gde se obezbeđuje da se u jednom trenutku čita najviše i/ili upisuje najviše jedan podatak. Poslednji uslov je neophodan kako bi se niz sintetisao u BRAM-u umesto u registrima. Širina jedne linije bafera se u ovom slučaju definiše parametrom **C_MAX_IMG_WIDTH** koji se zadaje prilikom instanciranja komponente. Ukoliko se za kreiranje linijskog bafera koristi komponenta RAM-based Shift Register potrebno je najpre uključiti ovaj IP blok u projekat odabirom odgovarajuće komponente iz IP kataloga. Nakon toga se otvara prozor sa podešavanjima. Dubinu memorije je potrebno podesiti na vrednost 512 što predstavlja maksimalnu širinu slike koja se obrađuje. Modul podžava varijabilnu širinu linije, pri čemu se ona kontroliše pomoću porta A na koji je potrebno dovesti vrednost poslednje kolone slike ($imgW-1$). Potrebno je omogućiti zaustavljanje upisa i čitanja iz linijskog bafera ukoliko na ulazu nema validnih podataka ili ukoliko izlaz nije spreman da prihvati nove podatke. Ova funkcionalnost se obezbeđuje uvođenjem clock enable (CE) signala. Podešavanje komponente IP modula je prikazano na slici 4. Nakon toga je potrebno generisati Output Products za instancirani modul. Odabrati opciju Global umesto Out-of-context. Templejt za instanciranje kreiranog IP-a u okviru VHDL komponente akceleratora se nalazi u fajlu: **ime_projekta.ip_user_files/ip/c_shit_ram_0/c_shift_ram_0.vho**. Za parametre komponente podešene kao na slici 4, za instanciranje bloka se može koristiti opis komponente prikazan na slici 5.



Slika 4. Podešavanje bloka RAM-based Shift Register

```

COMPONENT c_shift_ram_0
PORT (
  A : IN STD_LOGIC_VECTOR(8 DOWNTO 0);
  D : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
  CLK : IN STD_LOGIC;
  CE : IN STD_LOGIC;
  Q : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
);
END COMPONENT;

```

Slika 5. Instanciranje bloka RAM-based Shift Register